

STUDY GUIDE

SQL Server from definition to example

Stored procedures, functions, triggers, indexes, files, T-SQL, window functions, jobs, high availability, joins, and how SQL Server compares with other databases.

Prepared 28 September 2026

Contents

1. Stored procedure
2. Function and user-defined function
3. Trigger
4. Temporary table
5. Indexes — clustered, nonclustered, columnstore, CHARINDEX, PATINDEX
6. MDF, NDF, and LDF
7. NVARCHAR and all common data types
8. SQL syntax
9. T-SQL syntax
10. Window functions
11. Find duplicates
12. GROUP BY and HAVING
13. SQL Server Agent jobs
14. Log shipping
15. JDBC connection
16. Generate scripts in SSMS
17. SSMS working guide
18. Stored procedure optimization
19. Execution plan
20. Linked server and linked list
21. Magic tables
22. Replication
23. SQL Server, PostgreSQL, MySQL, Snowflake, Redshift, Azure SQL, Azure Synapse
24. DBMS and RDBMS
25. Codd's rules and Boyce-Codd normal form
26. ACID
27. View and materialized view
28. Local variables and global variables
29. Joins, join hints, and physical joins
30. MERGE
31. Common table expression
32. BULK INSERT

Two names from the original list are mapped here. **Pipe index** is treated as **PATINDEX** (pattern search). **Linked list** in a SQL Server syllabus is usually **linked server**; both are covered. **Boyce-Codd** and **Codd's rules** are both covered.

1. Stored procedure

A **stored procedure** is a named batch of T-SQL saved in the database and executed with `EXEC`. The engine can reuse an execution plan, you pass parameters, and you can grant permission on the procedure without granting direct table access.

Use it for business operations that run often: place an order, close a month, load a file, or return a shaped result set.

```
CREATE OR ALTER PROCEDURE dbo.usp_GetOrders
    @CustomerId int,
    @FromDate date = '2020-01-01'
AS
BEGIN
    SET NOCOUNT ON;

    SELECT OrderId, OrderDate, Amount
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
        AND OrderDate >= @FromDate
    ORDER BY OrderDate DESC;
END;
GO

EXEC dbo.usp_GetOrders @CustomerId = 42, @FromDate = '2026-01-01';
```

A procedure can contain `INSERT`, `UPDATE`, `DELETE`, transactions, error handling, and multiple result sets. It does not have to return a table. It can return an integer status with `RETURN` (often 0 means success).

```
CREATE OR ALTER PROCEDURE dbo.usp_CloseOrder
    @OrderId int
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRAN;

        UPDATE dbo.Orders
        SET Status = 'Closed', ClosedAt = SYSUTCDATETIME()
        WHERE OrderId = @OrderId
            AND Status = 'Open';

        IF @@ROWCOUNT = 0
            THROW 50001, 'Order was not open.', 1;

        COMMIT;
        RETURN 0;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        THROW;
    END CATCH
END;
```

2. Function and user-defined function

A **function** returns a value or a table and is used inside a statement: `SELECT`, `WHERE`, or a computed column. A **user-defined function (UDF)** is one you write. Built-in functions (`GETDATE`, `LEN`, `SUM`) ship with the engine.

Kind	Returns	Can change data?	Typical use
Scalar	One value	No	Format, calculate, classify a row
Inline table-valued	A <code>SELECT</code>	No	Reusable filtered or joined set. Best performance.
Multi-statement table-valued	A table you fill	No	Complex shaping. Often slower.

```
-- Scalar
CREATE OR ALTER FUNCTION dbo.fn_FullName
    (@First nvarchar(50), @Last nvarchar(50))
RETURNS nvarchar(101)
AS
BEGIN
    RETURN CONCAT(@First, N' ', @Last);
END;
GO

SELECT dbo.fn_FullName(N'Ada', N'Lovelace') AS FullName;

-- Inline table-valued (prefer this kind)
CREATE OR ALTER FUNCTION dbo.fn_OrdersFor (@CustomerId int)
RETURNS TABLE
AS
RETURN
(
    SELECT OrderId, OrderDate, Amount
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT * FROM dbo.fn_OrdersFor(42);
```

A procedure is executed with `EXEC`. A function is called inside SQL. A function must return something and cannot change permanent tables, start an explicit transaction, or use arbitrary side effects. Inline table-valued functions optimize like views. Scalar functions called once per row can hurt plans. SQL Server 2019 and later can inline many scalar UDFs, and an inline TVF is still the safer design.

3. Trigger

A **trigger** is code the engine runs automatically when a table, view, database, or server event occurs.

DML triggers are `AFTER INSERT`, `AFTER UPDATE`, `AFTER DELETE`, or `INSTEAD OF`. An `INSTEAD OF` trigger replaces the original write and is common on views.

```
CREATE OR ALTER TRIGGER dbo.tr_Orders_Audit
ON dbo.Orders
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.OrderAudit (OrderId, OldAmount, NewAmount, ChangedAt)
    SELECT d.OrderId, d.Amount, i.Amount, SYSUTCDATETIME()
    FROM inserted AS i
    JOIN deleted AS d ON d.OrderId = i.OrderId
    WHERE d.Amount <> i.Amount;
END;
```

`inserted` and `deleted` are the magic tables. They hold every row the statement affected, not one row at a time. Write triggers as set-based code.

Use a trigger for auditing, for a rule a constraint cannot express, or to keep a derived table in step. Prefer constraints, computed columns, and explicit procedures when those can do the job, because a trigger is invisible to the person writing the `UPDATE`.

DDL triggers fire on `CREATE`, `ALTER`, and `DROP`. Logon triggers fire when a session starts.

4. Temporary table

A **temporary table** lives in `tempdb` and disappears when the session, or the procedure that created it, ends.

Object	Scope	Name
Local temp table	This session	#Orders
Global temp table	All sessions, until the creator disconnects and nobody else is using it	##Orders
Table variable	The batch or procedure	@Orders

```
CREATE TABLE #TopCustomers
(
    CustomerId int PRIMARY KEY,
    TotalAmount decimal(18,2) NOT NULL
);

INSERT INTO #TopCustomers (CustomerId, TotalAmount)
SELECT CustomerId, SUM(Amount)
FROM dbo.Orders
GROUP BY CustomerId
HAVING SUM(Amount) > 10000;

SELECT c.CustomerName, t.TotalAmount
FROM #TopCustomers AS t
JOIN dbo.Customers AS c ON c.CustomerId = t.CustomerId;

DROP TABLE #TopCustomers; -- also dropped at end of session
```

`#temp` tables can have indexes and statistics, so the optimizer can estimate large sets. Table variables historically had a one-row estimate. Newer SQL Server can defer compilation for them, and for a large intermediate set a `#temp` table is still the usual choice. A global `##temp` table is visible to every session and is rarely the right tool.

5. Indexes

An **index** is a structure that lets SQL Server find rows without reading the whole table. The classic structure is a B-tree: sorted keys, with leaf pages holding either the full row or a pointer to it.

Clustered index

The **clustered index** is the table's physical sort order. Leaf pages are the data rows. A table can have one clustered index. A primary key creates one by default unless you specify `NONCLUSTERED`.

```
CREATE CLUSTERED INDEX CX_Orders_OrderDate
ON dbo.Orders (OrderDate, OrderId);
```

A good clustered key is narrow, unique, and ever-increasing, such as an `OrderId` identity, or a date plus an id. A random `uniqueidentifier` as the clustered key inserts new rows into the middle of the leaf and splits pages.

Nonclustered index

A **nonclustered index** is a separate sorted copy of selected columns, plus a pointer back to the clustered key (or a row id if the table is a heap). A table can have many.

```
CREATE NONCLUSTERED INDEX IX_Orders_Customer
ON dbo.Orders (CustomerId, OrderDate)
INCLUDE (Amount, Status);
```

Key columns (`CustomerId`, `OrderDate`) support seek and order. `INCLUDE` columns sit at the leaf so the query can be covered without going back to the base table. That extra trip is a **key lookup**.

A **heap** is a table with no clustered index. Nonclustered indexes then point at a row identifier. Most OLTP tables should have a clustered index.

Columnstore index

A **columnstore index** stores each column in its own compressed segments, in batches of about a million rows. Analytics queries that scan a few columns of a huge table read far less data than a rowstore B-tree.

```
-- Whole table stored as columns. Best for fact tables you mostly append to.
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales
ON dbo.FactSales;

-- Extra analytics index on a rowstore OLTP table.
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders_Analytics
ON dbo.Orders (OrderDate, CustomerId, Amount);
```

Rowstore answers "find this order." Columnstore answers "sum amount by month across 200 million rows." A clustered columnstore makes the columnstore the table itself. Deletes and updates are kept in a small rowstore delta store and a delete bitmap, then compressed later by the tuple mover.

CHARINDEX and PATINDEX

These are string functions, not index objects. They return the starting position of a match, or 0 when there is no match. PATINDEX is the pattern form that matches the phrase “pipe index” in the original topic list.

```
-- CHARINDEX: exact substring. Optional start position.
SELECT CHARINDEX(N'@', N'ada@example.com');           -- 4
SELECT CHARINDEX(N'ex', N'ada@example.com', 5);       -- 5

-- PATINDEX: pattern with % and _, same wildcards as LIKE
SELECT PATINDEX(N'%[0-9]%', N'INV-2048');             -- 5, first digit
SELECT PATINDEX(N'%example%', N'ada@example.com');    -- 5
```

CHARINDEX searches for a literal. PATINDEX searches for a pattern. Neither uses a B-tree to speed a search inside the string. A leading-wildcard LIKE or CHARINDEX against a column is usually a scan.

Other indexes

```
-- Unique
CREATE UNIQUE NONCLUSTERED INDEX UX_Customers_Email
ON dbo.Customers (Email);

-- Filtered: only rows that match the predicate
CREATE NONCLUSTERED INDEX IX_Orders_Open
ON dbo.Orders (CustomerId)
WHERE Status = 'Open';

-- Primary key (constraint plus index)
ALTER TABLE dbo.Orders
ADD CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (OrderId);
```

6. MDF, NDF, and LDF

SQL Server stores a database as files on disk.

Extension	Role	How many
MDF	Primary data file. Holds system tables and, by default, your data. The database cannot start without it.	Exactly one
NDF	Secondary data file. Extra room, often on another disk or in another filegroup.	Zero or more
LDF	Transaction log. Records every change so SQL Server can recover, roll back, and ship logs.	At least one

```
CREATE DATABASE Sales
ON PRIMARY
(
    NAME = Sales_Data,
    FILENAME = N'D:\Data\Sales.mdf',
    SIZE = 200MB, FILEGROWTH = 64MB
),
FILEGROUP Archive
(
    NAME = Sales_Archive,
    FILENAME = N'E:\Data\Sales_Archive.ndf',
    SIZE = 500MB, FILEGROWTH = 128MB
)
LOG ON
(
    NAME = Sales_Log,
    FILENAME = N'L:\Logs\Sales.ldf',
    SIZE = 100MB, FILEGROWTH = 64MB
);
```

Put large, cold tables on a filegroup whose files are NDF files. Put the log on its own fast disk. The log is written sequentially. Data files are read and written more randomly. Do not delete an MDF or LDF from disk while SQL Server has the database online. Detach, back up, or drop the database first.

```
SELECT name, type_desc, physical_name, size * 8 / 1024 AS size_mb
FROM sys.database_files;
```

7. NVARCHAR and the data types

nvarchar stores Unicode text in UTF-16. Each character uses 2 bytes. `nvarchar(100)` holds up to 100 characters. `nvarchar(max)` holds up to about 2 GB. Use `nvarchar` when names, addresses, or product text can contain characters outside one code page. Use `varchar` when you have confirmed the data is a single code page and the space savings matter.

Prefix Unicode literals with `N`, as in `N'München'`.

Integers

Type	Range	Bytes
bit	0, 1, or NULL	1 bit, packed
tinyint	0 to 255	1
smallint	−32,768 to 32,767	2
int	about ±2.1 billion	4
bigint	about $\pm 9 \times 10^{18}$	8

Exact and approximate numbers

`decimal(p,s)` and `numeric(p,s)` are exact. `p` is total digits (1–38). `s` is digits after the decimal. Use these for money when you want full control. `money` (8 bytes) and `smallmoney` exist and round to 4 decimal places. Many teams prefer `decimal(19,4)`.

`float(n)` and `real` (which is `float(24)`) are binary floating point. Do not use them for currency.

Date and time

Type	What it stores
date	Calendar date
time(p)	Time of day, precision 0–7
datetime2(p)	Date and time. Preferred for new work.
datetimeoffset(p)	Date, time, and a time-zone offset
datetime	Older type. About 3.33 ms accuracy, years 1753–9999.
smalldatetime	Minute accuracy, years 1900–2079

Text

Type	Character set	Length
char(n)	Code page	Fixed n
varchar(n), varchar(max)	Code page	Variable
nchar(n)	Unicode	Fixed n
nvarchar(n), nvarchar(max)	Unicode	Variable

`text`, `ntext`, and `image` are legacy. Use `varchar(max)`, `nvarchar(max)`, and `varbinary(max)`.

Binary and other types

Binary types are `binary(n)`, `varbinary(n)`, and `varbinary(max)`.

Type	Use
uniqueidentifier	GUID, 16 bytes
rowversion (alias timestamp)	Automatic 8-byte row version on each change. It is not a clock time.
xml	Typed or untyped XML, with optional XML indexes
json	Native JSON on newer engines. Otherwise store JSON in <code>nvarchar(max)</code> and use <code>OPENJSON</code> and <code>JSON_VALUE</code> .
sql_variant	Can hold almost any scalar. Awkward to query, index, and compare.
hierarchyid	A path in a tree
geometry, geography	Spatial data
vector	Embeddings, on very new SQL Server and Azure SQL builds

8. SQL syntax

These statements are the core of SQL. They work across SQL Server, PostgreSQL, and MySQL with small dialect differences.

```
-- Filter, project, sort
SELECT CustomerId, SUM(Amount) AS TotalAmount
FROM dbo.Orders
WHERE OrderDate >= '2026-01-01'
      AND Status IN ('Open', 'Shipped')
GROUP BY CustomerId
HAVING SUM(Amount) > 1000
ORDER BY TotalAmount DESC
OFFSET 0 ROWS FETCH NEXT 50 ROWS ONLY;

-- Change data
INSERT INTO dbo.Customers (CustomerName, Email)
VALUES (N'Ada', N'ada@example.com');

UPDATE dbo.Orders SET Status = 'Shipped' WHERE OrderId = 10;

DELETE FROM dbo.Orders WHERE OrderId = 10;

-- Define
CREATE TABLE dbo.Orders
(
    OrderId      int IDENTITY(1,1) NOT NULL
                CONSTRAINT PK_Orders PRIMARY KEY,
    CustomerId   int NOT NULL,
    OrderDate    date NOT NULL
                CONSTRAINT DF_Orders_OrderDate
                DEFAULT (CAST(SYSDATETIME() AS date)),
    Amount       decimal(18,2) NOT NULL,
    Status       varchar(20) NOT NULL,
    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerId) REFERENCES dbo.Customers (CustomerId),
    CONSTRAINT CK_Orders_Amount CHECK (Amount >= 0)
);

-- Remove
TRUNCATE TABLE dbo.StagingOrders; -- deallocates pages; no WHERE
DROP TABLE dbo.StagingOrders;
```

The logical order of a `SELECT` is: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, then `OFFSET/FETCH`. A `SELECT` alias is not yet in scope in `WHERE`. It is in scope in `ORDER BY`.

Predicates: `=`, `<>`, `<`, `>`, `<=`, `>=`, `IN`, `BETWEEN`, `LIKE`, `IS NULL`, `IS NOT NULL`, `EXISTS`, `AND`, `OR`, `NOT`.

`NULL` means unknown. `NULL = NULL` is unknown, so it does not pass a `WHERE` test. Compare with `IS NULL`.

9. T-SQL syntax

T-SQL is Microsoft's extension of SQL: variables, flow control, error handling, procedures, and system procedures. The SQL in the previous section is a subset of T-SQL.

```
DECLARE @CustomerId int = 42;
DECLARE @Total decimal(18,2);

SELECT @Total = SUM(Amount)
FROM dbo.Orders
WHERE CustomerId = @CustomerId;

IF @Total IS NULL
    PRINT 'No orders';
ELSE IF @Total > 10000
    PRINT 'High value';
ELSE
    PRINT 'Standard';

WHILE @CustomerId < 50
BEGIN
    SET @CustomerId += 1;
    IF @CustomerId = 45 CONTINUE;
    BREAK;
END

BEGIN TRY
    BEGIN TRAN;
    -- work
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    THROW;
END CATCH
```

```
-- Dynamic SQL, parameterized
DECLARE @sql nvarchar(max) =
    N'SELECT * FROM dbo.Orders WHERE CustomerId = @id';
EXEC sys.sp_executesql @sql, N'@id int', @id = 42;

-- Output from a DML statement
INSERT INTO dbo.Customers (CustomerName)
OUTPUT inserted.CustomerId, inserted.CustomerName
VALUES (N'Grace');

-- Exists test
IF OBJECT_ID(N'dbo.Orders', N'U') IS NOT NULL
    DROP TABLE dbo.Orders;

SET NOCOUNT ON;
SET XACT_ABORT ON;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Batches are separated by `GO`. `GO` is a client command, not a T-SQL keyword. Variables die at the end of the batch. `SET XACT_ABORT ON` rolls back the transaction on most runtime errors.

10. Window functions

A **window function** computes a value across a set of rows related to the current row, without collapsing those rows the way `GROUP BY` does.

```
SELECT
    OrderId,
    CustomerId,
    OrderDate,
    Amount,
    SUM(Amount) OVER (PARTITION BY CustomerId) AS CustomerTotal,
    ROW_NUMBER() OVER (
        PARTITION BY CustomerId ORDER BY OrderDate DESC
    ) AS RecentRank,
    LAG(Amount) OVER (
        PARTITION BY CustomerId ORDER BY OrderDate
    ) AS PrevAmount,
    LEAD(Amount) OVER (
        PARTITION BY CustomerId ORDER BY OrderDate
    ) AS NextAmount,
    SUM(Amount) OVER (
        PARTITION BY CustomerId
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
FROM dbo.Orders;
```

Families:

- **Ranking:** `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE(n)`
- **Aggregate over a window:** `SUM`, `COUNT`, `AVG`, `MIN`, `MAX`
- **Offset:** `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`
- **Distribution:** `PERCENT_RANK`, `CUME_DIST`, `PERCENTILE_CONT`, `PERCENTILE_DISC`

`PARTITION BY` resets the window per group. `ORDER BY` defines sequence inside the partition. The frame (`ROWS` or `RANGE`) defines which neighbors participate. When you supply `ORDER BY` and no frame, the default is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`, which includes peers that tie on the order key. Prefer `ROWS` when you want a physical running total.

`RANK` skips numbers after a tie (1, 1, 3). `DENSE_RANK` does not (1, 1, 2). `ROW_NUMBER` never ties. If its `ORDER BY` is not unique, which row gets which number is arbitrary until you add a unique tie-break such as the primary key.

11. Find duplicates

Duplicates mean more than one row shares the columns you care about.

```
-- Which values are duplicated
SELECT Email, COUNT(*) AS Cnt
FROM dbo.Customers
GROUP BY Email
HAVING COUNT(*) > 1;

-- Show every duplicate row
WITH d AS
(
    SELECT
        CustomerId,
        Email,
        COUNT(*) OVER (PARTITION BY Email) AS Cnt
    FROM dbo.Customers
)
SELECT CustomerId, Email
FROM d
WHERE Cnt > 1
ORDER BY Email, CustomerId;
```

To keep one row and delete the rest, name the keeper explicitly. Run the `SELECT` form first and look at the rows.

```
WITH ranked AS
(
    SELECT
        CustomerId,
        ROW_NUMBER() OVER (
            PARTITION BY Email ORDER BY CustomerId
        ) AS rn
    FROM dbo.Customers
)
DELETE FROM ranked
WHERE rn > 1;
```

SQL Server allows deleting through a CTE when the CTE maps to one base table.

12. GROUP BY and HAVING

`GROUP BY` collapses rows into groups. Every selected column is either in the `GROUP BY` or inside an aggregate (`SUM`, `COUNT`, `MIN`, `MAX`, `AVG`, `STRING_AGG`).

`WHERE` filters rows before grouping. `HAVING` filters groups after aggregation.

```
SELECT CustomerId, Status, COUNT(*) AS Orders, SUM(Amount) AS Total
FROM dbo.Orders
WHERE OrderDate >= '2026-01-01'
GROUP BY CustomerId, Status
HAVING SUM(Amount) > 5000
      AND COUNT(*) >= 3;
```

`GROUPING SETS`, `ROLLUP`, and `CUBE` add subtotals.

```
SELECT CustomerId, Status, SUM(Amount) AS Total
FROM dbo.Orders
GROUP BY GROUPING SETS
(
    (CustomerId, Status),
    (CustomerId),
    ()
);
```

13. SQL Server Agent jobs

A **job** is scheduled work run by **SQL Server Agent**, a separate Windows service. A job has steps (T-SQL, PowerShell, SSIS, or an operating-system command), schedules, and notifications.

In SSMS: SQL Server Agent, then Jobs, then New Job.

```
USE msdb;
GO

EXEC dbo.sp_add_job
    @job_name = N'Nightly order archive',
    @enabled = 1;

EXEC dbo.sp_add_jobstep
    @job_name = N'Nightly order archive',
    @step_name = N'Archive',
    @subsystem = N'TSQL',
    @database_name = N'Sales',
    @command = N'EXEC dbo.usp_ArchiveClosedOrders;',
    @retry_attempts = 2,
    @retry_interval = 5;

EXEC dbo.sp_add_schedule
    @schedule_name = N'Every night 01:00',
    @freq_type = 4,           -- daily
    @freq_interval = 1,
    @active_start_time = 010000; -- HHMMSS

EXEC dbo.sp_attach_schedule
    @job_name = N'Nightly order archive',
    @schedule_name = N'Every night 01:00';

EXEC dbo.sp_add_jobserver
    @job_name = N'Nightly order archive';
```

Agent must be running. Jobs live in `msdb`. History is on the job's History page and in `msdb.dbo.sysjobhistory`. Set a step's failure action to quit the job reporting failure when a failed step should stop the rest of the job.

14. Log shipping

Log shipping keeps a warm copy of a database on another server by a repeating cycle:

1. **Backup job** on the primary: `BACKUP LOG` on a schedule, often every few minutes.
2. **Copy job** on the secondary: copy those transaction-log backup files.
3. **Restore job** on the secondary: `RESTORE LOG ... WITH NORECOVERY`, or `STANDBY` if you want read-only access between restores.

The secondary stays in a restoring state. `RESTORE DATABASE ... WITH RECOVERY` brings it online and ends the shipping chain. An optional monitor server records backup and restore age so you can alert when the secondary falls behind.

Use log shipping for disaster recovery and for a reporting copy that can be minutes behind. It is not synchronous. **Always On availability groups** are the newer high-availability feature: synchronous or asynchronous replicas, readable secondaries, and automatic failover. Log shipping remains simple and works on editions that do not include availability groups.

Setup is under the database, Tasks, Ship Transaction Logs, or through `msdb` procedures such as `sp_add_log_shipping_primary_database`.

15. JDBC connection

JDBC is Java's database API. The Microsoft driver class is

`com.microsoft.sqlserver.jdbc.SQLServerDriver`. Add the `mssql-jdbc` jar to the project.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class SqlExample {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:sqlserver://localhost:1433;"
            + "databaseName=Sales;"
            + "encrypt=true;"
            + "trustServerCertificate=true;"; // local dev only

        try (Connection conn = DriverManager.getConnection(
            url, "app_user", "secret");
            PreparedStatement ps = conn.prepareStatement(
                "SELECT OrderId, Amount FROM dbo.Orders "
                + "WHERE CustomerId = ?")) {

            ps.setInt(1, 42);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    System.out.println(
                        rs.getInt("OrderId") + " "
                        + rs.getBigDecimal("Amount"));
                }
            }
        }
    }
}
```

Use `PreparedStatement` and placeholders. Concatenating user input into SQL is how SQL injection happens. For production, install a real TLS certificate and remove `trustServerCertificate`. An application should take connections from a pool such as HikariCP. Opening `DriverManager` once per request is fine for a demo and expensive in a service.

```
try (CallableStatement cs = conn.prepareCall(
    "{call dbo.usp_GetOrders(?, ?)}")) {
    cs.setInt(1, 42);
    cs.setDate(2, java.sql.Date.valueOf("2026-01-01"));
    try (ResultSet rs = cs.executeQuery()) { /* read rows */ }
}
```

16. Generate scripts in SSMS

SSMS can emit `CREATE` and `INSERT` script from objects that already exist.

One object. Right-click the table, procedure, or view, then Script object as, then CREATE To, then New Query Editor Window.

Many objects or data.

1. Right-click the database, then Tasks, then Generate Scripts.
2. Choose specific objects or the whole database.
3. Open Advanced.
4. Set Types of data to script to Schema only, Data only, or Schema and data.
5. Set Script Indexes, Script Foreign Keys, and Script DROP and CREATE as needed.
6. Save to a file, the clipboard, or a new query window.

To script a change you are about to make in the table designer, enable Tools, Options, Designers, Auto generate change scripts, or use Generate Change Script before you save. A script of every row is not a recovery plan for a large database. A backup is.

17. SSMS working guide

SQL Server Management Studio is the desktop client for SQL Server and Azure SQL.

Connect. Provide the server name and authentication (Windows or a SQL login), and optionally a database. Recent connections stay on the login dialog. Registered Servers stores groups of servers.

Object Explorer. The tree holds databases, tables, views, programmability (procedures and functions), security, and SQL Server Agent. Right-click is where Script, Design, Properties, and Tasks live. Object Explorer Details (F7) is a grid of the current node.

Query window. New Query. The database dropdown sets the database for the batch. F5 executes. Highlight a portion to run only that text. **GO** splits batches.

Results. Results to Grid is Ctrl+D. Results to Text is Ctrl+T. The Messages tab shows row counts and **PRINT**. Sorting a column header sorts the client-side grid. It does not change the query.

Execution plan. Ctrl+L is the estimated plan. The query is not run. Ctrl+M includes the actual plan on the next run. Read operators from right to left and top to bottom. A thicker arrow means more rows.

Other tools:

- **Activity Monitor** shows who is running, what they are waiting on, and expensive queries.
- **Database diagrams** picture foreign keys. Useful for learning a small schema.
- **Template Explorer** holds parameterized script stubs.
- **SQLCMD mode** (Query menu) lets a script use **:setvar**.
- **IntelliSense:** Ctrl+Space. Refresh the cache with Ctrl+Shift+R when a new table is missing from completion.
- **Options** control the result grid, the sound on completion, and whether the designer may save changes that recreate a table.

Security in the UI: server Security, then Logins, is the instance login. Each database has Security, then Users, which maps a login into that database, plus database roles. A login connects to the instance. A user is that login inside one database.

18. Stored procedure optimization

A procedure is fast when the statements inside it are fast and when the cached plan matches the parameter values you usually pass.

1. `SET NOCOUNT ON` at the top, so a row-count message is not sent after every statement.
2. Write set-based SQL. A cursor or a `WHILE` over every row is the usual reason a procedure is slow.
3. Keep predicates SARGable. The column stands alone on one side of the comparison so an index can seek.

```
-- Seekable
WHERE OrderDate >= '2026-01-01'
    AND OrderDate < '2026-02-01'

-- Not seekable: a function wraps the column
WHERE YEAR(OrderDate) = 2026
WHERE CONVERT(date, CreatedAt) = '2026-09-01'
```

4. Cover the query. A nonclustered index whose keys match `WHERE` and `JOIN`, and whose `INCLUDE` list holds the selected columns, avoids key lookups.
5. Watch parameter sniffing. SQL Server compiles using the first parameter values it sees, then reuses that plan. A plan sniffed for a tiny customer can be a poor plan for a huge customer.

```
-- A local variable hides the parameter from the first sniff
DECLARE @Id int = @CustomerId;
SELECT Amount FROM dbo.Orders WHERE CustomerId = @Id;

-- Recompile this statement on every execution
SELECT Amount
FROM dbo.Orders
WHERE CustomerId = @CustomerId
OPTION (RECOMPILE);
```

`OPTION (RECOMPILE)` fits a statement whose best plan changes sharply with the parameter and that does not run thousands of times per second. `OPTION (OPTIMIZE FOR UNKNOWN)` asks for an average plan.

6. Keep transactions short. Change the rows, commit, then do slow work such as email or file I/O outside the transaction.
7. Avoid `SELECT *`. It blocks covering indexes and breaks clients when a column is added.
8. Match data types. A `varchar` parameter compared with an `nvarchar` column can cause a scan. A scalar UDF in `WHERE` can force row-by-row execution.
9. Read the actual plan and `SET STATISTICS IO, TIME ON`. Logical reads tell you how much data you touched.

```
SET STATISTICS IO, TIME ON;
EXEC dbo.usp_GetOrders @CustomerId = 42;
```

19. Execution plan

The **execution plan** is the tree of operators the optimizer chose. Each operator has an estimated cost. The estimate stays a guess until you run the query. The **actual plan** adds real row counts.

Operator	Meaning
Index Seek	Walked the B-tree to the matching keys. This is what you want for a selective filter.
Index Scan or Clustered Index Scan	Read the index or the table. Reasonable when the query needs a large fraction of the rows.
Key Lookup	After a nonclustered seek, fetched extra columns from the clustered index. Many lookups mean you should INCLUDE those columns or select fewer of them.
Hash Match	Builds a hash table on one input and probes with the other. Fits large unsorted joins and aggregates.
Merge Join	Both inputs are sorted on the join key and walked together. Efficient when that order already exists.
Nested Loops	For each outer row, find matches on the inner side. Best when the outer side is small and the inner side has an index.
Sort	An explicit sort. Expensive when it spills to tempdb.
Parallelism	Work split across threads, then gathered.

Plan warnings to take seriously: a missing-index suggestion (a hint, not an order), an implicit conversion, a spill to `tempdb`, and estimated rows far from actual rows. The last of those is stale statistics or a sniffed parameter.

```
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;
```

20. Linked server and linked list

A **linked server** is a named connection from this SQL Server instance to another data source: another SQL Server, Oracle, Excel, or an ODBC source. You query it with a four-part name:

`server.database.schema.table`.

```
EXEC sp_addlinkedserver
    @server = N'REPORTING',
    @srvproduct = N'',
    @provider = N'MSOLEDBSQL',
    @datasrc = N'report-host';

EXEC sp_addlinkedsrvlogin
    @rmtsrvname = N'REPORTING',
    @useself = N'False',
    @locallogin = NULL,
    @rmtuser = N'report_reader',
    @rmtpassword = N'secret';

SELECT OrderId, Amount
FROM REPORTING.Sales.dbo.Orders
WHERE OrderDate >= '2026-01-01';
```

`OPENQUERY` sends the SQL text to the remote engine so filtering happens there.

```
SELECT *
FROM OPENQUERY(REPORTING,
    'SELECT OrderId, Amount
    FROM Sales.dbo.Orders
    WHERE OrderDate >= ''2026-01-01''');
```

A four-part query can pull a remote table locally and filter afterward. `OPENQUERY` keeps the filter on the remote server.

A **linked list** is a programming structure: each node holds a value and a pointer to the next node. SQL tables are not linked lists. You model a chain with an adjacency list (`EmployeeId`, `ManagerId`) and walk it with a recursive common table expression.

21. Magic tables

`inserted` and `deleted` exist inside a trigger. The `OUTPUT` clause of `INSERT`, `UPDATE`, and `DELETE` exposes the same row images under those names.

Statement	inserted	deleted
INSERT	The new rows	Empty
DELETE	Empty	The old rows
UPDATE	The new version	The old version

They have the same columns as the base table, and they contain every row the statement affected. A trigger that assumes exactly one row is wrong under a multi-row `UPDATE`.

```
CREATE OR ALTER TRIGGER dbo.tr_BlockAmountDrop
ON dbo.Orders
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS
    (
        SELECT 1
        FROM inserted AS i
        JOIN deleted AS d ON d.OrderId = i.OrderId
        WHERE i.Amount < d.Amount
    )
        THROW 50002, 'Amount cannot decrease.', 1;
END;
```

22. Replication

Replication copies data and schema changes from a Publisher through a Distributor to one or more Subscribers.

Type	What moves	Typical use
Snapshot	A full refresh of the articles	Small tables, an occasional copy
Transactional	Each committed change, in order, near real time	A reporting copy or an integration feed
Peer-to-peer	Transactional, with more than one writable node	Scale-out writes. Conflicts are a design problem you own.
Merge	Changes from both sides, with conflict resolvers	Occasionally connected clients

An **article** is one table, procedure, or view. A **publication** is the set of articles. The log reader agent reads the publisher's transaction log. The distribution agent applies commands at the subscriber.

Transactional replication is not a backup. The subscriber diverges if someone writes to it directly, and the distributor database needs its own maintenance. For high availability, availability groups or log shipping are the usual tools. Use replication when the subscriber should hold a subset of tables, a different set of indexes, or a filtered copy.

23. Database engines compared

Engine	What it is	Strength	How you scale
SQL Server	Microsoft's relational engine, including Linux builds. T-SQL, Agent, columnstore, replication, availability groups.	OLTP and mixed workloads, SSMS, Windows integration.	Bigger hardware, readable replicas. Sharding is an application design.
PostgreSQL	Open-source relational engine. Rich types, extensions such as PostGIS, strong standard SQL, MVCC.	Complex queries, extensibility, license cost.	Bigger hardware, streaming replicas. Citus and similar projects distribute it.
MySQL	Open-source relational engine. InnoDB is the storage engine that matters. Very common behind web applications.	Straightforward web OLTP and a large hosting ecosystem.	Read replicas. Sharding through the application or Vitess.
Azure SQL	Microsoft's managed SQL Server: a single database, an elastic pool, or a managed instance. Application T-SQL is largely the same.	SQL Server without operating a VM. Backups, patching, and high availability are part of the service.	Service tier: vCores, serverless, or Hyperscale.
Snowflake	A cloud warehouse. Storage and compute are separate. It uses tables and ANSI-style SQL aimed at analytics.	Many teams sharing data, warehouses you can suspend, and time travel.	Change warehouse size, or run more warehouses at once.
Amazon Redshift	AWS columnar warehouse, historically based on PostgreSQL and ParAccel. Massively parallel across nodes.	Large analytical scans inside AWS.	Add nodes. RA3 separates storage. Concurrency scaling covers bursts.
Azure Synapse	Azure's analytics platform. Dedicated SQL pools are the distributed warehouse (formerly Azure SQL Data Warehouse). Serverless SQL queries files in the data lake. Spark and pipelines sit beside the SQL.	Lakehouse-style analytics on Azure, mixing SQL and Spark.	Scale dedicated-pool DWUs, or pay per query on serverless SQL.

Application transactions — orders, accounts, locking, stored procedures — belong on SQL Server, Azure SQL, PostgreSQL, or MySQL. If you already write T-SQL and want it managed, use Azure SQL. Analysts scanning billions of rows, loading files, and serving many dashboards belong on Snowflake, Redshift, or Synapse. Those three warehouses are a weak fit for thousands of single-row OLTP updates per second. SQL Server and PostgreSQL are a weak fit as the only engine for a petabyte data lake scanned by dozens of BI tools.

Dialect differences that show up immediately:

- Limiting rows: SQL Server uses `OFFSET/FETCH` or `TOP`. PostgreSQL and MySQL use `LIMIT`.
- Auto-number: `IDENTITY` or a sequence in SQL Server; `GENERATED AS IDENTITY` in PostgreSQL; `AUTO_INCREMENT` in MySQL.
- Strings: `nvarchar` and the `N` prefix are a SQL Server habit. PostgreSQL `varchar` is already Unicode.

- Upsert: `MERGE` in SQL Server; `INSERT ... ON CONFLICT` in PostgreSQL; `INSERT ... ON DUPLICATE KEY UPDATE` in MySQL.

24. DBMS and RDBMS

A **DBMS** (database management system) is any engine that stores data and lets you define, query, and protect it. That includes relational engines, document stores, key-value stores, and graph stores.

An **RDBMS** is a DBMS whose model is relations: tables of rows and columns, keys, and a query language based on relational algebra, which is what SQL implements. SQL Server, PostgreSQL, MySQL, and Azure SQL are RDBMSs. Snowflake, Redshift, and Synapse dedicated SQL pools use the same table model and give up some OLTP behavior, such as rich constraint enforcement and high-rate single-row updates.

25. Codd's rules and Boyce-Codd

E. F. Codd's rules, numbered 0 through 12, describe what it means for a product to be relational. No major product scores a perfect 12. They remain the definition people mean by a relational system.

0. The system manages data through its relational capabilities.
1. All information is represented as values in tables.
2. Every value is addressable by table name, primary key, and column name.
3. Nulls are systematic, and they are distinct from an empty string or zero.
4. The schema itself lives in tables (the catalog) and is queried with the same language.
5. One language covers definition, queries, authorization, integrity, and transactions.
6. Views that are theoretically updatable can be updated through the system.
7. Insert, update, and delete work on sets, not only on one row at a time.
8. Application programs keep working when physical storage changes (files, indexes).
9. Application programs keep working when tables are split or joined, provided views preserve the information.
10. Integrity rules live in the catalog, not only in application code.
11. Distributing data across nodes does not change how the application addresses it.
12. A lower-level interface is not allowed to bypass the integrity rules.

Boyce-Codd normal form (BCNF) is a table-design rule, stricter than third normal form. A table is in BCNF when every determinant is a candidate key. If column A determines column B, then A must be a key. This removes a class of update anomalies.

A classic table that can be in third normal form and still violate BCNF is `StudentSubjectTeacher (Student, Subject, Teacher)` where each subject has many students, each teacher teaches one subject, and each subject is taught by one teacher in this model. `Teacher` determines `Subject`, and `Teacher` is not a candidate key. Split it into `TeacherSubject (Teacher, Subject)` and `StudentTeacher (Student, Teacher)`.

26. ACID

ACID is the promise of a transaction.

Property	Meaning	In SQL Server
Atomicity	Every statement in the transaction commits, or none of them remain.	BEGIN TRAN, COMMIT, and ROLLBACK. The log makes rollback possible.
Consistency	A committed transaction leaves the database satisfying its constraints.	PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE. Triggers can add further rules.
Isolation	Concurrent transactions do not observe each other's half-finished work, to the degree the isolation level promises.	READ COMMITTED is the default. Also READ UNCOMMITTED, REPEATABLE READ, SNAPSHOT, and SERIALIZABLE.
Durability	After commit, the change survives a crash.	Commit waits until the log record is written to the LDF.

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;  
BEGIN TRAN;  
UPDATE dbo.Accounts SET Balance = Balance - 100 WHERE AccountId = 1;  
UPDATE dbo.Accounts SET Balance = Balance + 100 WHERE AccountId = 2;  
COMMIT;
```

SNAPSHOT requires **ALLOW_SNAPSHOT_ISOLATION** on the database. **SNAPSHOT** and **READ COMMITTED** **SNAPSHOT** keep row versions in **tempdb** so a reader sees the last committed version and does not wait on the writer's lock.

27. View and materialized view

A **view** is a stored `SELECT`. Querying the view runs that `SELECT`. The view does not store rows unless you index it.

```
CREATE OR ALTER VIEW dbo.v_OpenOrders
AS
SELECT o.OrderId, c.CustomerName, o.OrderDate, o.Amount
FROM dbo.Orders AS o
JOIN dbo.Customers AS c ON c.CustomerId = o.CustomerId
WHERE o.Status = 'Open';
```

A simple view on one table, with the key preserved and no aggregates, is updatable. A view with joins or `GROUP BY` generally is not, unless you write an `INSTEAD OF` trigger.

A **materialized view** stores the query result and refreshes it. PostgreSQL has `CREATE MATERIALIZED VIEW` and `REFRESH MATERIALIZED VIEW`. Snowflake maintains materialized views as data changes.

SQL Server's equivalent is an **indexed view**: a view with a unique clustered index. The index stores the joined or aggregated rows, and the engine maintains it when the base tables change. Requirements are strict: `SCHEMABINDING`, deterministic functions, `COUNT_BIG(*)` when you aggregate, and session options such as `ANSI_NULLS ON` and `QUOTED_IDENTIFIER ON`.

```
CREATE OR ALTER VIEW dbo.v_CustomerTotals
WITH SCHEMABINDING
AS
SELECT CustomerId,
       COUNT_BIG(*) AS Cnt,
       SUM(Amount) AS Total
FROM dbo.Orders
GROUP BY CustomerId;
GO

CREATE UNIQUE CLUSTERED INDEX IX_v_CustomerTotals
ON dbo.v_CustomerTotals (CustomerId);
```

Enterprise Edition can use this index even when the query never names the view. On Standard Edition you often query the view directly and add `WITH (NOEXPAND)`.

28. Local variables and global variables

Local variables belong to the batch or procedure. You declare them. They disappear when that batch ends.

```
DECLARE @Name nvarchar(100);
DECLARE @Count int = 0;

SET @Name = N'Ada';
SELECT @Count = COUNT(*) FROM dbo.Orders;
```

`SELECT @Count = COUNT(*)` is a normal assignment. `SELECT @x = Col FROM Table` against many rows leaves `@x` holding one of those values, and which one is not defined. Use an aggregate or `TOP (1)` with `ORDER BY` when you mean one specific value.

What people call **global variables** in SQL Server are system functions spelled with `@@`. You do not create them. They report session or server state.

Function	Reports
@@VERSION	The build of this instance
@@SERVERNAME	The server name
@@SPID	This session id
@@ROWCOUNT	Rows affected by the immediately previous statement
@@ERROR	Error number of the previous statement, or 0. Prefer TRY/CATCH and ERROR_NUMBER().
@@IDENTITY	Last identity value generated on this session, by any table. Prefer SCOPE_IDENTITY() for your own statement's scope.
@@TRANCOUNT	How many transactions are open on this session

```
UPDATE dbo.Orders SET Status = 'Shipped' WHERE OrderId = 10;
SELECT @@ROWCOUNT AS UpdatedRows;
```

There is no user-defined variable shared across sessions. To share a value, write a one-row table. `sp_set_session_context` stores a value for the current session only.

```
EXEC sp_set_session_context @key = N'TenantId', @value = 7;
SELECT SESSION_CONTEXT(N'TenantId');
```

29. Joins, join hints, and physical joins

Logical joins

```
-- Inner: only rows that match on both sides
SELECT o.OrderId, c.CustomerName
FROM dbo.Orders AS o
INNER JOIN dbo.Customers AS c
    ON c.CustomerId = o.CustomerId;

-- Left: every order; customer columns are NULL when unmatched
SELECT o.OrderId, c.CustomerName
FROM dbo.Orders AS o
LEFT JOIN dbo.Customers AS c
    ON c.CustomerId = o.CustomerId;

-- Cross: every row of A paired with every row of B
SELECT d.DayDate, s.ShiftName
FROM dbo.Calendar AS d
CROSS JOIN dbo.Shifts AS s;

-- Self join: the table matched to itself, always with two aliases
SELECT e.EmployeeName AS Employee, m.EmployeeName AS Manager
FROM dbo.Employees AS e
LEFT JOIN dbo.Employees AS m
    ON m.EmployeeId = e.ManagerId;
```

A cross join of 10,000 rows by 10,000 rows produces 100 million rows. Use it for small combinations such as dates times shifts. `FULL OUTER JOIN` keeps unmatched rows from both sides. `RIGHT JOIN` is a left join written the other way around. Most teams write `LEFT JOIN` so the preserved table is always the first one.

Physical joins

- **Nested loops:** for each outer row, find inner matches. Fast when one side is small and the other is indexed.
- **Hash:** hash the build input, probe with the other. Fast for large inputs that are not already ordered. Needs memory.
- **Merge:** both sides sorted on the join key, then advanced together like two sorted lists. Fast when indexes already provide that order.

Join hints

A hint forces the algorithm. Use one after you have measured a bad plan and confirmed the forced shape stays stable.

```
SELECT o.OrderId, c.CustomerName
FROM dbo.Orders AS o
INNER HASH JOIN dbo.Customers AS c
    ON c.CustomerId = o.CustomerId;
```

```
SELECT o.OrderId, c.CustomerName
FROM dbo.Orders AS o
INNER LOOP JOIN dbo.Customers AS c
    ON c.CustomerId = o.CustomerId;
```

```
SELECT o.OrderId, c.CustomerName
FROM dbo.Orders AS o
INNER MERGE JOIN dbo.Customers AS c
    ON c.CustomerId = o.CustomerId;
```

Query-wide forms are `OPTION (HASH JOIN)`, `OPTION (LOOP JOIN)`, `OPTION (MERGE JOIN)`, and `OPTION (FORCE ORDER)`. A supporting index is the better long-term fix when one exists. Hints have to be revisited when data volumes change.

30. MERGE

MERGE inserts, updates, and deletes in one statement by comparing a target table with a source.

```
MERGE dbo.Customers AS t
USING dbo.StagingCustomers AS s
    ON t.Email = s.Email
WHEN MATCHED AND t.CustomerName <> s.CustomerName THEN
    UPDATE SET CustomerName = s.CustomerName
WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerName, Email)
    VALUES (s.CustomerName, s.Email)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

The source can be a table, a view, or a derived table from **VALUES** or a CTE. Put an extra predicate on **WHEN MATCHED** so unchanged rows are not rewritten.

- The join must identify at most one source row per target row. SQL Server raises an error if one target row matches several source rows.
- Under concurrency, **MERGE** has known race conditions. A staged batch load is the comfortable use. A single-row upsert is often written as an explicit transaction that takes **UPDLOCK** and **HOLDLOCK**.
- **OUTPUT \$action, inserted.*, deleted.*** shows what the statement did to each row.

31. Common table expression

A **CTE** is a named subquery that lives for one statement. It makes the query easier to read. It is not a temp table, and you cannot index it by itself. SQL Server may spool the result.

```
WITH monthly AS
(
    SELECT CustomerId, SUM(Amount) AS Total
    FROM dbo.Orders
    WHERE OrderDate >= '2026-01-01'
    GROUP BY CustomerId
)
SELECT c.CustomerName, m.Total
FROM monthly AS m
JOIN dbo.Customers AS c ON c.CustomerId = m.CustomerId
WHERE m.Total > 1000;
```

Several CTEs can share one **WITH**, separated by commas. A recursive CTE walks a tree or a graph that is a tree in practice.

```
WITH org AS
(
    SELECT EmployeeId, ManagerId, EmployeeName, 0 AS Lvl
    FROM dbo.Employees
    WHERE ManagerId IS NULL

    UNION ALL

    SELECT e.EmployeeId, e.ManagerId, e.EmployeeName, o.Lvl + 1
    FROM dbo.Employees AS e
    JOIN org AS o ON e.ManagerId = o.EmployeeId
)
SELECT EmployeeId, EmployeeName, Lvl
FROM org
OPTION (MAXRECURSION 100);
```

The anchor member runs once. The recursive member runs until it returns no rows, or until **MAXRECURSION** stops it. The default limit is 100.

32. BULK INSERT

BULK INSERT loads a file that the SQL Server service account can read. The path is on the server, or on a share that account can open. It is not a path on your laptop unless that path is visible to the service.

```
BULK INSERT dbo.StagingOrders
FROM N'D:\import\orders.csv'
WITH
(
    FORMAT = 'CSV',
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    TABLOCK,
    CODEPAGE = '65001' -- UTF-8
);
```

TABLOCK, a heap or an empty clustered table, and the simple or bulk-logged recovery model can minimally log the load. Constraints and indexes still apply unless you disable them on purpose and validate the data afterward.

Other loaders: the **bcp** and **sqlcmd** command-line tools; **OPENROWSET(BULK ...)** when the file should appear as a rowset inside **INSERT...SELECT**; SSIS or Azure Data Factory when the pipeline needs conversions, rejects, and a schedule.

```
INSERT INTO dbo.StagingOrders (OrderId, Amount)
SELECT OrderId, Amount
FROM OPENROWSET
(
    BULK 'D:\import\orders.csv',
    FORMATFILE = 'D:\import\orders.xml'
) AS r;
```

How the pieces fit

A request arrives through JDBC or through SSMS. It runs a procedure. The procedure uses local variables, maybe a CTE or a `#temp` table, joins with a seek on a nonclustered index, and writes rows into the MDF or NDF while the change is hardened in the LDF. A trigger sees `inserted` and `deleted` and writes an audit row. An Agent job later archives old rows. Log shipping or replication carries the log or the articles to another server. An analytical copy may sit in a columnstore index, an indexed view, or an entirely different engine such as Synapse or Snowflake.

The highest-leverage next study set is execution plans, index design (the clustered key plus covering nonclustered indexes), and parameter sniffing. Those three explain most cases of a procedure that is slow.